

9jaRide: project overview and pitch guide

Written from a full read of the repository. Everything under "What exists" is in the code today. Anything that is not built or not verified is listed under "What is not done", so you never claim more than the product does. Numbers about market size, users or revenue are **not** in the repo, so none are invented here. Fill those in from your own research.

1. The one-liner

9jaRide is a ride-hailing platform for Nigeria: a rider app, a driver app, and an operations portal, built around safe rides, honest pricing, and money that is tracked to the kobo.

Shorter, for introductions: *"Uber-style rides for Lagos, with the safety, fleet and payments tools a local operator actually needs."*

2. The 30-second pitch

"Booking a ride in Nigeria should feel safe and predictable. 9jaRide gives riders a simple app to book now or schedule ahead, see a fare range up front, track the driver live, and hit an SOS button if something is wrong. Drivers get an app built for real conditions: it keeps working through weak signal and is careful with battery. Behind both is an operations portal where staff onboard and verify drivers, manage fares, handle safety alerts, and see every naira that moves. We also support something most ride apps don't: drivers who don't own a car can get one through the platform or drive a business's vehicle, with repayments handled automatically."

3. The problem and the opportunity

State these as the reasons the product is shaped the way it is. They are design choices visible in the code, not market statistics.

- **Trust and safety.** Riders worry about who is driving them. The product answers with a verified-driver process (ID check, documents, review), driver and vehicle details shown before pickup, trip sharing guidance, and an SOS flow that is stored before it is confirmed and escalates if nobody responds.
- **Unreliable connectivity and cheap phones.** The driver app queues location offline and uploads it later, and the BATTERY notes show deliberate work to cut requests and GPS use.
- **Cash and payment friction.** Riders can pay cash or from an in-app wallet funded through Paystack. The money system is built so that nothing is double-credited and nothing disappears.
- **Many drivers can't afford a car.** Fleet and vehicle-plan features let a driver drive a car that someone else owns, or buy one in instalments from earnings.
- **Operators need control.** Fares, commissions, promos, cancellation rules and home-screen content are all changed from a portal, and sensitive changes need a second person to approve.

4. What exists

Three products sharing one backend.

4.1 Rider app (Android, Kotlin and Jetpack Compose)

- Sign up and sign in with a phone code, light and dark mode.
- **Home screen:** top 40% live map of the rider's location; bottom 60% one continuous panel with "Where are you going?", schedule a ride, recent locations, ride options (Regular, Comfort, Send Package), a "For You" carousel, an "Invite and Earn ₦1,000" card, announcements and safety tips.
- Search an address or pin it on the map, then choose a ride category and see a **fare range** and the payment method (cash or wallet).
- Request a ride, see the searching state with a cancel option, then see the driver's name, rating, vehicle, plate, live position and arrival time.
- In-trip view, **SOS button**, trip complete screen with receipt and rating.
- **Scheduled rides:** a one-off ride or a weekly series (2 to 12 weeks).
- Trip history and detail, wallet with activity and Paystack top-up, help, account deletion guidance.

4.2 Driver app (Android, Kotlin and Jetpack Compose)

- Phone sign-in, forced-update check, and a guided **onboarding**: choose how you get a car, enter details, upload documents, NIN check, settlement (payout) details, then wait for staff review. If staff ask for changes, the driver fixes only the named parts.
- Go online, receive ride offers, accept, navigate through arrive, start and complete.
- **Background location** through a foreground service, with an offline queue that uploads batches, battery-optimisation guidance per phone brand, and resume after reboot.
- Earnings, wallet, payouts, profile. Drivers using someone else's car must agree to the arrangement before going online.
- A **demo mode** with sample data and a scripted ride, so the app can be shown without a server.

4.3 Admin portal (React, Vite and TypeScript)

Staff roles are support, finance and admin, and the server enforces them. - **Operations:** dashboard, live map of drivers and trips (refreshes every 5 seconds), trips with fare snapshot and refund or adjust, customers, drivers, vehicles. - **Safety Center:** SOS queue and detail with acknowledge, assign, notes, timeline and resolve. - **Driver onboarding queue:** review documents, approve, request changes on named parts, or reject. - **Finance:** wallet overview, driver payouts, adjustments and refunds, reconciliation with payment exceptions, revenue by day, category and method, and a full ledger. - **Setup:** trip fees with version history, revenue setup (commission and who shares it), cancellation policy, asset types, promo codes, **home-screen cards** that control what riders see on Home. - **Fleet:** businesses and owners, a central vehicle list, vehicle assignment to drivers, vehicle payment plans and instalments, owner and business staff access. - **Team and audit:** invite staff, change roles, activity logs, CSV exports, ratings, support desk. - Business partners can sign in and manage only their own vehicles.

4.4 Backend (NestJS, PostgreSQL with PostGIS, Valkey/Redis)

A modular monolith with 27 database migrations and these modules: auth, rides, dispatch, fare, ledger, payments, safety, admin, console, settings, catalog, fleet, vehicle plans, stakeholders, promo, support, files, NIN verification, team, home content and app config.

5. How a ride works

1. **Quote.** The rider picks pickup and drop-off. The server returns a fare range per category, computed from versioned, approved rates.
2. **Request.** The ride is created once (idempotency keys stop double-booking). A wallet ride holds the top of the quoted range.
3. **Dispatch.** Nearby available drivers are offered the ride. The database guarantees **one ride never goes to two drivers and one driver never gets two offers**.
4. **Pickup and trip.** The driver accepts, arrives, starts, and the rider watches the car live. Location is recorded for the trip.
5. **Complete.** The fare is calculated, the receipt lines add up to the total exactly, and the fare snapshot is immutable.
6. **Money.** Commission, tax and driver earnings are posted to a double-entry ledger. Both sides rate each other.
7. **Safety checks.** If a driver's reported distance is far above the recorded route, staff are flagged. The fare is never changed automatically.

6. Why it is different (your selling points)

Use these, because each one is backed by the code.

1. **Money you can audit.** An append-only, double-entry ledger in integer kobo. Balances are derived, never edited. Posting is idempotent. Refunds, credits and payouts need a second approver, and the same person can never approve their own request. This is enforced in the service and in database constraints.
2. **Safety is a system.** SOS is confirmed only after it is stored, escalates if unacknowledged after 60 seconds, and has a timeline and staff workflow. Drivers reach the map only through a reviewed application, with NIN verification and expiring document checks.
3. **Built for Nigerian conditions.** Offline location queue, battery-conscious design, Lagos time zone, naira and kobo throughout, Paystack for payments, NIN identity check, and battery-optimisation guidance for Tecno, Infinix, itel, Samsung and Xiaomi style phones.
4. **Fleet and vehicle financing.** Three driver arrangements: own car, get a car through 9jaRide on an instalment plan, or drive someone else's car. Deductions from earnings go to the owner automatically and stop when the vehicle is paid off.
5. **Operator control without code changes.** Fares, commission split, cancellation rules, promos and rider home cards are all managed in the portal. Rate changes are versioned and need a second admin's approval.
6. **Scheduled and weekly rides.** Priced when dispatch starts, with wallet money held then, and a missed ride is cancelled rather than started late.
7. **Quality discipline.** Unit, component, API integration (real Postgres and Redis) and browser end-to-end tests, plus a real-money safety model tested against a fake payment provider.

7. Business model (as the product is built to support)

These are mechanisms present in the code. The actual percentages and prices are settings, not commitments.

- **Commission** on each trip, configurable in Revenue Setup (seeded at 12%, taken on the fare excluding a ₦30 tax line; to be confirmed with an accountant).
- **Platform vehicle plans:** vehicles sold to drivers in instalments, collected from earnings.
- **Business partners and owners:** a share of driver earnings goes to the vehicle owner, with the platform managing the process.
- **Stakeholder payouts:** the commission can be shared among defined parties, each paid through an approved payout.
- **Growth levers:** promo codes, referral and invite rewards (the rider Invite and Earn card), and scheduled and package rides.

8. Technology, for technical questions

Layer	Choice
Rider and driver apps	Kotlin, Jetpack Compose, three Gradle modules (<code>core</code> , <code>rider</code> , <code>driver</code>) sharing theme, API client, map and formatting
Backend	NestJS (Node 20, TypeScript), PostgreSQL 16 with PostGIS, Valkey/Redis for live state, JWT auth with rotating refresh tokens
Admin portal	React 18, Vite, TypeScript, Leaflet maps
Payments	Paystack, with webhook verification, hourly reconciliation and a payout approval flow
Hosting	<code>render.yaml</code> blueprint for API, database, cache and the portal
Size	Roughly 13,000 lines of backend code, 4,800 of admin portal and 9,300 of Android, 27 migrations and 44 test files

Notable engineering decisions: dispatch locking enforced by a database unique index, long-poll (held-open requests) instead of constant polling to save battery and server load, immutable fare snapshots, and append-only history for ledger, ratings and vehicle payments.

9. What is not done (be upfront about this)

Investors and partners respect honesty, and these are all known and written down in the repo.

- **Test-mode sign-in.** The hosted copy uses a fixed code (`0000`). An SMS provider must be connected before launch.
- **Payments not live.** Paystack is not configured on the test copy, so top-ups and payouts don't run there, and the live Paystack has never been exercised.
- **No push notifications** (Firebase). Apps currently use held-open requests. Closed apps can't be reached.
- **Maps, routing and place search** use OpenStreetMap, the public OSRM server and Nominatim, which are for development only. Paid providers are needed.
- **No two-factor for staff** and custom roles are not built, so the three roles are fixed.
- **NIN verification** runs against a stub until a real provider is connected.
- **iOS** is not started. Only Android exists.
- **Not field-tested** on battery-aggressive phones, and no load or failure-injection testing has been done.

- **Cancellation fees, driver cancellation notice, referral reward payout** and some bonus rules are deliberately left until the business decides them.
- **Commission and tax treatment** (with or without tax) need an accountant's confirmation. Fare numbers in the repo are starting values, not final prices.
- **Tokens** are kept in app preferences and should move to the Android Keystore.

10. Roadmap you can credibly state

Before a pilot: connect an SMS provider and set live OTP; switch Paystack on and test with real small amounts; connect a real NIN provider; replace development map services; field-test on target phones; add push notifications.

Pilot: a single city (Lagos, which the code is already tuned for), a small vetted driver group, cash and wallet payments, daily review in the portal.

After the pilot: staff two-factor, iOS apps, referral and bonus programmes, load testing, and further cities.

11. A five-minute demo script

1. **Portal first (1 min).** Show the live map and dashboard. Explain that this is how an operator runs the service.
2. **Driver onboarding (1 min).** Show an application, the document checks and the NIN result, then approve it.
3. **Rider app (1.5 min).** Open Home and point out the 40/60 layout. Search a destination, show the fare range, request the ride, and let the driver's demo accept.
4. **Driver app (30 s).** Show the offer, accept, and the trip.
5. **Safety (30 s).** Press SOS in the rider app, then show it arriving in the Safety Center with a timeline.
6. **Money (30 s).** Open the trip's receipt, then the ledger entries for the same trip.

Tip: the driver app has a demo mode, so a flaky connection won't ruin the demo. Practise with the backend running locally.

12. Questions you should expect, with honest answers

How is this different from Uber or Bolt? Don't claim to out-scale them. The honest angle is a local operator toolkit: vehicle financing and fleet partners, a fully auditable money system, a safety workflow, and an operations portal an operator controls without developers.

Is it live? Not yet. It's a working product with a test deployment. The remaining launch steps (SMS, live payments, real maps, push) are listed in section 10.

How do you make money? Commission per trip, plus vehicle plan and fleet arrangements. Rates are settings in the portal.

How do you keep riders safe? Verified drivers (reviewed application, NIN check, expiring documents), driver and vehicle details before pickup, an SOS flow with escalation, and staff tooling to suspend people immediately.

What stops fraud on money? Ledger entries can't be edited, every manual money movement needs a second person, payments are verified against Paystack for the exact amount, and daily reconciliation catches missed webhooks.

What happens in bad network? The driver app stores location on the phone and uploads in batches. Duplicate uploads are safe.

What's the team and traction? Not in the repo. Add your own.

13. Gaps in this document for you to fill

- Your name, role and the story of why you started this
- Market size, target city and competitor facts, from your own research
- Pricing you actually intend to charge, once confirmed with an accountant
- Pilot plans, partners, any real users or drivers, and funding needs
- Screenshots of the apps and portal for slides (the code can't generate these here)

14. Where to look in the repo

Topic	File
Backend rules and decisions	backend/README.md
Android apps	android/README.md
Admin portal screens	admin/README.md
Hosting a test copy	DEPLOY.md
Testing and debugging	TESTING.md
Battery behaviour	BATTERY.md